COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE031	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 17, 2026	**DATE SUBMITTED:**	Oct 24, 2026
NAME:	ANDREW DOLOSO JR. RALPH FLORENZ DUE?AS JAREL DAVE EGAR	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

DOLOSO

Task 1

```
#Code Here

import pandas as pd
import numpy as np
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(42)
months = pd.date_range("2024-01-01", periods=10, freq="MS")
classes = ["Class A", "Class B", "Class C", "Class D"]
subjects = ["Math", "Science", "English"]

df = pd.DataFrame({
    "Month": np.random.choice(months, 300),
    "Class": np.random.choice(classes, 300),
    "Subject": np.random.choice(subjects, 300),
    "Score": np.random.randint(50, 100, 300),
    "StudyHours": np.random.randint(1, 15, 300)
})
df["Result"] = np.where(df["Score"] >= 75, "Passed", "Needs Improvement")

score_by_class = df.groupby("Class", as_index=False)["Score"].mean()
result_counts = df["Result"].value_counts().reset_index()
result_counts.columns = ["Result", "Count"]
score_trend = df.groupby("Month", as_index=False)["Score"].mean().sort_values("Month")

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=("Average Score by Class", "Pass vs Needs Improvement", "Monthly Average Score Trend"),
    specs=[[{"type": "bar"}, {"type": "pie"}], [{"colspan": 2}, None]]
)

fig.add_trace(go.Bar(x=score_by_class["Class"], y=score_by_class["Score"].round(1),
                     text=score_by_class["Score"].round(1), textposition="outside",
                     marker_color="#2A9D8F", name="Avg Score"), row=1, col=1)

fig.add_trace(go.Pie(labels=result_counts["Result"], values=result_counts["Count"],
                     marker_colors=["#2A9D8F", "#E76F51"], hole=0.4,
                     textinfo="label+percent", name="Results"), row=1, col=2)

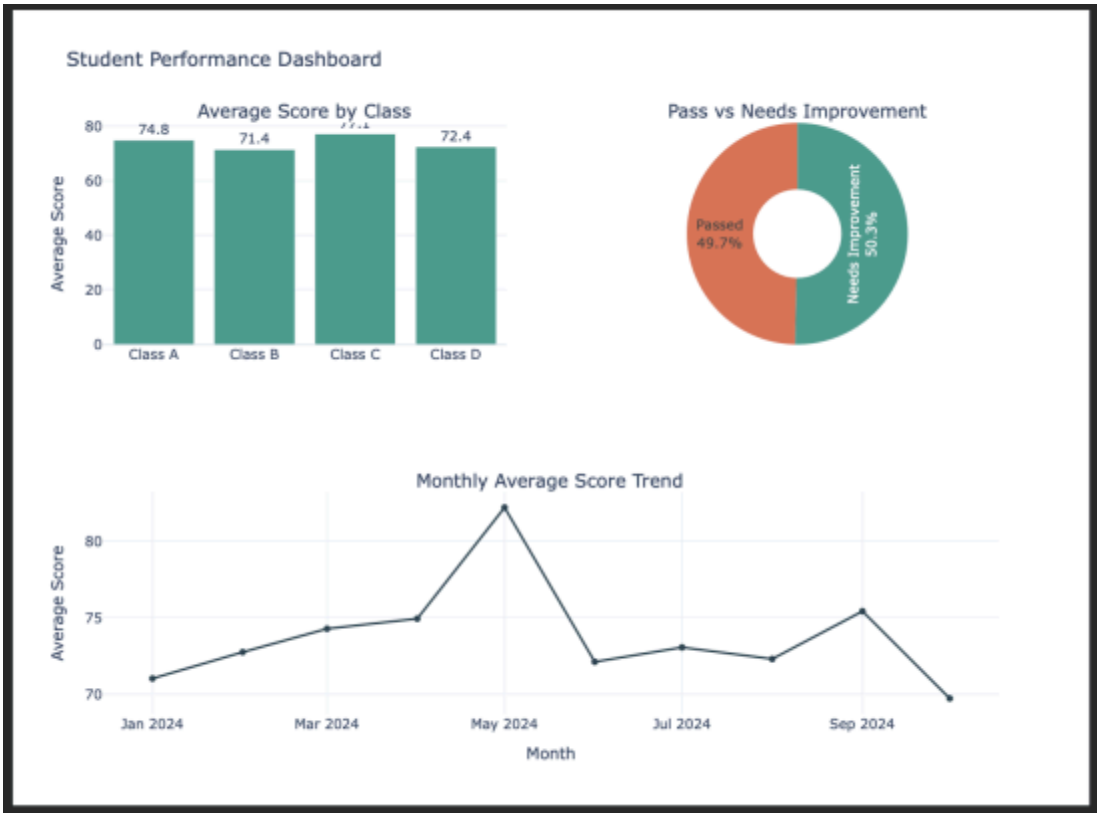
fig.add_trace(go.Scatter(x=score_trend["Month"], y=score_trend["Score"],
                         mode="lines+markers", line=dict(color="#264653"),
                         name="Monthly Trend"), row=2, col=1)

fig.update_yaxes(title_text="Average Score", row=1, col=1)
fig.update_yaxes(title_text="Average Score", row=2, col=1)
fig.update_xaxes(title_text="Month", row=2, col=1)
fig.update_layout(height=700, width=950, template="plotly_white",
                  title_text="Student Performance Dashboard", showlegend=False)
fig.show()
```

COMPUTER ENGINEERING

QUEZON CITY

DOLOSO



Task 2

```
#Code Here

import plotly.express as px
import time
from IPython.display import clear_output

# Interactive dashboard: filterable by subject (click the legend)
avg_scores = df.groupby(["Class", "Subject"], as_index=False)["Score"].mean().round(1)
fig = px.bar(avg_scores, x="Class", y="Score", color="Subject", barmode="group",
             title="Interactive Dashboard: Average Score by Class and Subject")
fig.show()

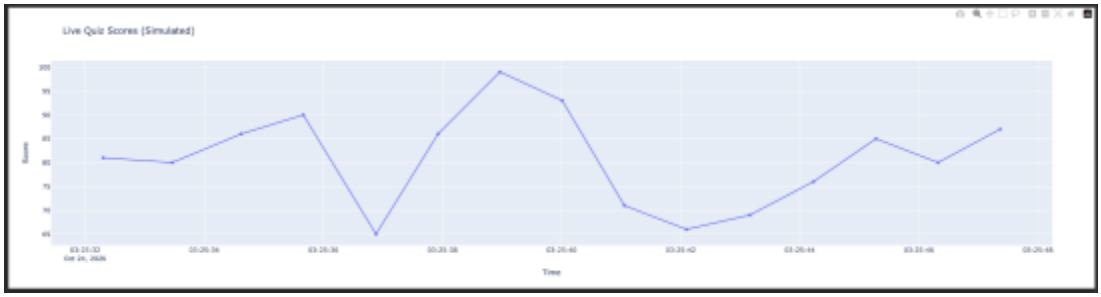
# Dynamic update: simulated real-time quiz scores
times, scores = [], []
for i in range(15):
    times.append(pd.Timestamp.now())
    scores.append(np.random.randint(60, 100))
    live = px.line(x=times, y=scores, markers=True, title="Live Quiz Scores (Simulated)",
                   labels={"x": "Time", "y": "Score"})
    clear_output(wait=True)
    live.show()
    time.sleep(1)
```

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

DOLOSO



DUENAS

Task 1

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

np.random.seed(42)
dates = pd.date_range("2024-01-01", periods=12, freq="M")
stores = ["Store A", "Store B", "Store C", "Store D"]
categories = ["Electronics", "Clothing", "Groceries"]

df = pd.DataFrame({
    "Month": np.random.choice(dates, 200),
    "Store": np.random.choice(stores, 200),
    "Category": np.random.choice(categories, 200),
    "Sales": np.random.randint(1000, 10000, 200),
    "Profit": np.random.randint(100, 2000, 200)
})

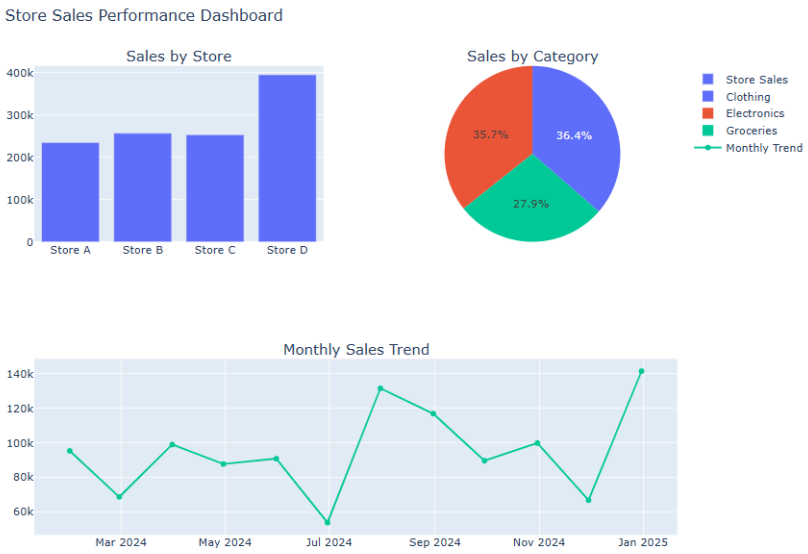
sales_by_store = df.groupby("Store", as_index=False)["Sales"].sum()
sales_by_cat = df.groupby("Category", as_index=False)["Sales"].sum()
sales_trend = df.groupby("Month", as_index=False)["Sales"].sum().sort_values("Month")

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=("Sales by Store", "Sales by Category", "Monthly Sales Trend"),
    specs=[[{"type": "bar"}, {"type": "pie"}], [{"colspan": 2}, None]]
)

fig.add_trace(go.Bar(x=sales_by_store["Store"], y=sales_by_store["Sales"], name="Store Sales"), row=1, col=1)
fig.add_trace(go.Pie(labels=sales_by_cat["Category"], values=sales_by_cat["Sales"], name="Category Sales"), row=1, col=2)
fig.add_trace(go.Scatter(x=sales_trend["Month"], y=sales_trend["Sales"], mode="lines+markers", name="Monthly Trend"), row=2, col=1)

fig.update_layout(height=700, width=950, title_text="Store Sales Performance Dashboard")
fig.show()
```

```
/tmp/ipykernel_429/523150443.py:8: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.
  dates = pd.date_range("2024-01-01", periods=12, freq="M")
```



COMPUTER ENGINEERING

QUEZON CITY

DUENAS

Task 2

```
import time
from IPython.display import clear_output

fig_interactive = px.bar(
    df,
    x="Store",
    y="Sales",
    color="Category",
    barmode="group",
    title="Interactive Dashboard: Sales by Store and Category",
    hover_data=["Profit"]
)

cat_names = list(df["Category"].unique())
buttons = [
    dict(
        label="All",
        method="update",
        args=[{"visible": [True] * len(cat_names)}, {"title": "Sales by Store - All Categories"}]
    )
]

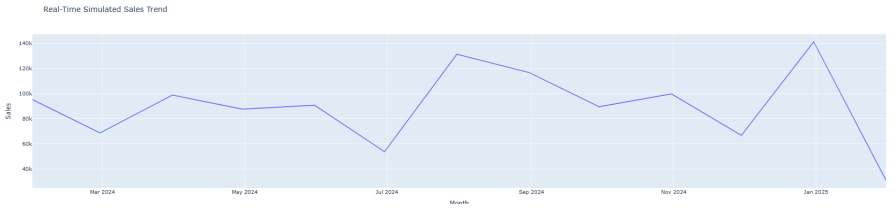
for i, cat in enumerate(cat_names):
    vis = [False] * len(cat_names)
    vis[i] = True
    buttons.append(
        dict(
            label=cat,
            method="update",
            args=[{"visible": vis}, {"title": f"Sales by Store - {cat}"}]
        )
    )

fig_interactive.update_layout(
    updatemenus=[dict(active=0, buttons=buttons, x=0.1, y=1.15)]
)
fig_interactive.show()

for i in range(5):
    new_data = pd.DataFrame({
        "Month": [pd.Timestamp("2025-01-31")],
        "Store": [np.random.choice(stores)],
        "Category": [np.random.choice(categories)],
        "Sales": [np.random.randint(1000, 10000)],
        "Profit": [np.random.randint(100, 2000)]
    })
    df = pd.concat([df, new_data], ignore_index=True)

    sales_trend = df.groupby("Month", as_index=False)["Sales"].sum().sort_values("Month")
    fig_dynamic = px.line(sales_trend, x="Month", y="Sales", title="Real-Time Simulated Sales Trend")

    clear_output(wait=True)
    fig_dynamic.show()
    time.sleep(1)
```



TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

EGAR

Task 1

```
[14]
✓ 0s
    # sample data set for people who uses spotify and youtube
    np.random.seed(42)
    months = pd.date_range("2024-01-01", periods=12, freq="M")
    regions = ["North", "South", "East", "West"]
    products = ["Spotify", "Youtube"]

    data = {
        "Month": np.random.choice(months, 200),
        "Region": np.random.choice(regions, 200),
        "Application": np.random.choice(products, 200),
        "Users": np.random.randint(5000, 20000, 200),
    }

    df = pd.DataFrame(data)
    df.head()

    /tmp/ipykernel_3360/3811622297.py:3: FutureWarning:

    'M' is deprecated and will be removed in a future version, please use 'ME' instead.
```

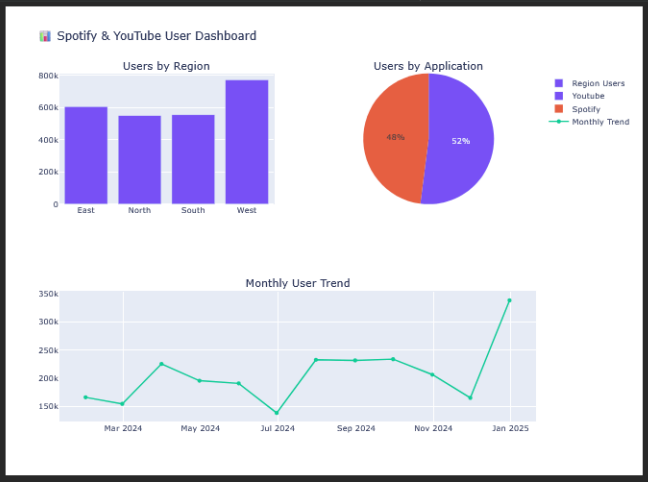
	Month	Region	Application	Users
0	2024-07-31	West	Youtube	17237
1	2024-04-30	West	Spotify	14435
2	2024-11-30	East	Spotify	10188
3	2024-08-31	East	Youtube	16310
4	2024-05-31	South	Spotify	18296

```
# Aggregated views
users_by_region = df.groupby("Region", as_index=False)["Users"].sum()
users_by_application = df.groupby("Application", as_index=False)["Users"].sum()
application_trend = df.groupby("Month", as_index=False)["Users"].sum().sort_values("Month")

# Create dashboard layout
fig = make_subplots(rows=2, cols=2,
                    subplot_titles=("Users by Region", "Users by Application", "Monthly User Trend"),
                    specs=[[{"type": "bar"}, {"type": "pie"}],
                           [{"colspan": 2}, None]])

# Add charts
fig.add_trace(go.Bar(x=users_by_region["Region"], y=users_by_region["Users"], name="Region Users"), row=1, col=1)
fig.add_trace(go.Pie(labels=users_by_application["Application"], values=users_by_application["Users"], name="Application Users"), row=1, col=2)
fig.add_trace(go.Scatter(x=application_trend["Month"], y=application_trend["Users"], mode="lines+markers", name="Monthly Trend"), row=2, col=1)

fig.update_layout(height=700, width=950, title_text="📊 Spotify & YouTube User Dashboard")
fig.show()
```



TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

EGAR

Task 2



SUPPLEMENTARY

```python
from IPython.display import clear_output, display
import ipywidgets as widgets
import pandas as pd
import plotly.express as px

# Student dataset with updated student names
df = pd.DataFrame(
    {
        "Student": ["Alex", "Brooke", "Chris", "Diana", "Ethan"],
        "Subject": ["Math", "Math", "Science", "Science", "English"],
        "Grade": [90, 78, 88, 75, 95],
        "Attendance": [95, 80, 90, 75, 98],
        "Week": [1, 2, 3, 4, 5],
    }
)

# Interactive subject dropdown widget
subject = widgets.Dropdown(
    options=["All"] + sorted(df["Subject"].unique().tolist()),
    value="All",
    description="Subject:",
)

output = widgets.Output()


def dashboard(change=None):
    with output:
        clear_output(wait=True)

        # Filter dataset based on widget selection
        data = (
            df if subject.value == "All" else df[df["Subject"] == subject.value]
        )

        # Print summary metrics
        print("Average Grade:", round(data["Grade"].mean(), 2))
        print("Average Attendance:", round(data["Attendance"].mean(), 2), "%")

        # Visualization 1: Average grades by subject
        grades = data.groupby("Subject", as_index=False)["Grade"].mean()
        fig1 = px.bar(
            grades,
            x="Subject",
            y="Grade",
            title="Average Grades by Subject",
            color="Subject",
        )
        fig1.show()

        # Visualization 2: Attendance by week
        attendance = df.groupby("Week", as_index=False)["Attendance"].mean()
        fig2 = px.line(
            attendance,
            x="Week",
            y="Attendance",
            markers=True,
            title="Attendance by Week",
        )
        fig2.show()

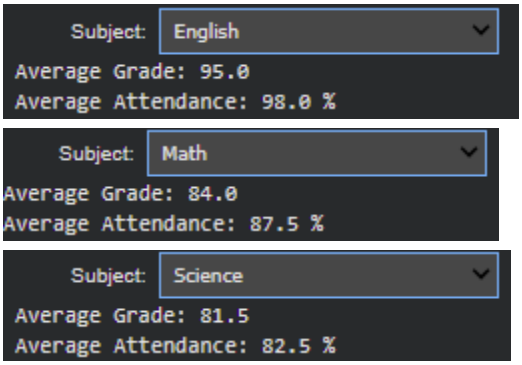
        # Visualization 3: Student performance trend
        fig3 = px.line(
            df,
            x="Week",
            y="Grade",
            markers=True,
            title="Student Performance Trend",
            color="Student",
        )
        fig3.show()


# Link widget to update function and render
subject.observe(dashboard, names="value")
display(subject, output)
dashboard()
```

COMPUTER ENGINEERING

QUEZON CITY

SUPPLEMENTARY



CONCLUSION / LEARNING / ANALYSIS:

Doloso :
In this activity, We learned how to create an interactive data dashboard using Python functions such as Pandas, numpy ,Plotly Express, and some ipywidgets. By using sample student data, I was able to display average grades, attendance percentages, and student performance, through graphs and summary metrics. We also learned how to use a dropdown menu to filter data by subject and update the displayed results. This activity helped us understand the importance of clarity, simplicity, and interactivity in presenting information. I realized that dashboards can help users analyze data, monitor performance, identify trends, and make better decisions. Although the program uses sample data and does not automatically update from an external data source, it demonstrates how Python can be used to organize and visualize information effectively.
In Conclusion, this activity improved our understanding of data analysis and visualization and showed us how it is useful in developing systems for monitoring performance and supporting decision-making.

Duenas :
In this HOA, we learned what makes a data dashboard work well. A good dashboard should be easy to read, simple to use, let users click around, and help people make choices. We also learned how dashboards are grouped by how often they get new data, how people use them, and their main job in a company (such as long-term planning, studying details, or daily tasks). We built a dashboard using practice data to display different numbers. This HOA showed us that a dashboard's design must match how its data updates, how people use it, and the main goal it serves.

Egar: This Hands on Activity has taught us about creating interactive data dashboards through Python programming tools including Pandas, NumPy, Plotly Express, and ipywidgets libraries. By analyzing the sample student data, my groupmates and I have been able to show the mean grade, attendance, and performance of the students through charts and summary measures. We also included a drop-down list to allow users to select filters and update the visualizations of the dashboard. Through this assignment, we understood that an effective data dashboard must be simple and easy to understand for allowing users to explore and

COMPUTER ENGINEERING

QUEZON CITY

CONCLUSION / LEARNING / ANALYSIS:
analyze data to make better decisions. We have realized that data dashboards can be classified according to their frequency of updates, interaction methods, and purposes. These purposes may include strategic planning, detailed analysis, or operational purposes among others.

COLABS
DOLOSO
Doloso_Hands_On_Activity_12___Data_Dashboard_Taxonomies-1.ipynb
DUENAS
Duenas_Hands_On_Activity_12___Data_Dashboard_Taxonomies-1.ipynb
EGAR
Hands_On_Activity_12___Data_Dashboard_Taxonomies-1.ipynb